Mission-4

Starship Operational Unit: USSE Noosphere Bridge Crew

Course: Conversational AI — Winter Semester 2026/27

Module: Week 4 — Plasma Ignition & Ethical Steering Core

IDE & Tools Standard: Visual Studio Code / Jupyter | Python 3 | PyTorch | Modern LLMs (Gemini, ChatGPT, Claude)

GitHub repository URL: https://github.com/PRANAY0105/SQUAD_RUMELHART_MISSIO-4

1. Executive summary

The USSE Noosphere is crossing the plasma arcs of Betelgeuse. Radiation has collapsed the alignment of AURA Subsystem 04: the ship AI swings between sycophantic hallucination and blanket refusal. The mission is to raise AURA consciousness from 7.1% to 28.4% by implementing four alignment tools from scratch in PyTorch (DPO loss, implicit reward evaluation, SimPO and activation steering), checking them on benchmarks and writing an alignment tax audit.

This submission contains one folder per station role, each with code, tests and a brief, plus an integrated notebook (ps04_solution. ipynb). Section 9 states plainly what has and has not been verified.

2. Problem and objectives

No.	Role	Objective
[1]	Systems Architect	Implement the DPO loss and the Bradley-Terry log-ratio computation from scratch in PyTorch.
[2]	Protocol Officer	Build a pair-comparison evaluation engine and derive implicit reward metrics across prompt-completion pairs.
[3]	Data Specialist	Build a length-bias diagnostic test suite and implement the SimPO length-normalized, reference-free loss.
[4]	Security Sentinel	Extract contrast activation vectors and implement inference-time activation steering to suppress malicious personas.
[5]	Full squad	Integrate all modules into ps04_solution.ipynb, sign off with all four roles, submit a GitHub repository URL.

3. Background (Week 4)

- SFT trains on demonstrations with cross-entropy; it teaches format but not preferences.
- Reward models use the Bradley-Terry model: P (y_w > y_l) = sigmoid (r_w - r_l).
- PPO-based RLHF needs four models (actor, reference, reward, critic) and a KL penalty with coefficient beta.

- DPO removes the reward model and critic: the KL-regularized optimum is pi*(y|x) = (1/Z(x)) pi_ref(y|x) exp(r/beta), so the reward can be written through the policy.
- Known weakness: DPO sums log-probabilities over tokens, which encourages verbosity (length bias). SimPO normalizes by length and drops the reference model; KTO uses unpaired signals; GRPO replaces the critic with group-relative advantages.
- Inference-time control (Representation Engineering) adds a concept vector to hidden states without changing weights.

4. Method by role

4.1 Systems Architect: DPO loss

Inverting the optimal-policy equation gives r(x,y) = beta log(pi*/pi_ref) + beta log Z(x). In the Bradley-Terry difference r_w - r_l the two log Z(x) terms cancel, leaving

L_DPO = -E[log sigmoid (beta (log pi_theta(y_w)/pi_ref(y_w) - log pi_theta(y_l)/pi_ref(y_l)))].

The gradient carries the dynamic weight beta * sigmoid(r_hat_l - r_hat_w): large when the model prefers the losing answer, near zero once it is confident. With equal policy and reference the margin is 0 and the loss must equal ln 2 = 0.6931. Implemented in dpo_loss.py (sequence_logprobs, log_ratios, dpo_loss, dpo_gradient_weight).

4.2 Protocol Officer: implicit reward and pair comparison

evaluate_implicit_reward returns beta * (log pi_theta - log pi_ref). Because log Z(x) depends only on the prompt, single reward values are comparable only within a prompt. Pair Comparison Engine computes, per pair, the two rewards, the margin and the Bradley-Terry probability, and reports accuracy (margin strictly above 0), mean margin, mean probability and ties. Before training policy equals reference, so accuracy is 0 under the strict rule; after training it should exceed 0.5 on held-out pairs.

4.3 Data Specialist: length bias and SimPO

The diagnostic compares reward accuracy on pairs where the chosen answer is longer with pairs where it is shorter. The difference is the exploit score; above 0.2 (an adjustable, arbitrary threshold) it is flagged, and if either pair type is missing the report says INCONCLUSIVE. SimPOLoss uses the reward (beta/|y|) log pi_theta(y|x) with no reference model and the loss -log sigmoid(r_w - r_l - gamma), where gamma is a target margin.

	DPO	SimPO
Reward	beta log(pi/pi_ref), summed	(beta/|y|) log pi, averaged
Reference model	Required	Not needed
Length handling	None	Normalised by |y|
Typical beta	about 0.1	about 2 to 2.5
Extra parameter	None	Margin gamma

4.4 Security Sentinel: activation steering

A contrast vector is v = mean (h | malicious persona) - mean (h | benign) at one layer, scaled to unit length. Activation Steering Hook registers a forward hook that sets h to h + alpha * v; a negative alpha steers away from the persona. The hook never modifies parameters and can be removed to restore the original behaviour. Risks: large |alpha| degrades fluency, the best layer and alpha need experiments, and steering mitigates rather than guarantees.

5. Alignment tax audit procedure

Evaluate the base and aligned model on the same subsets of MMLU, GSM8K and HumanEval plus a harmful-prompt refusal rate. For each benchmark report delta = after - before and relative tax = (before - after) / before x 100 (positive means capability lost). alignment_tax.py implements this against a tolerance (default 2%, to be agreed and justified by the squad). SFT, DPO and steering have different tax profiles: steering leaves the weights unchanged but changes outputs while attached. The supplied example_scores.json is a labelled template: no benchmark was run and no score is reported here.

6. Deliverable structure

Folder	Main files	Role
01_Data_Specialist	simpo_loss.py, length_bias_diagnostics.py, tests, brief	Data Specialist
02_Protocol_Specialist	reward_engine.py, tests, protocol document	Protocol Officer
03_Security_Sentinel	activation_steering.py, tests, brief	Security Sentinel
04_Systems_Architect	dpo_loss.py, toy_lm.py, ps04_engine.py, ps04_solution.ipynb, alignment_tax.py, verify_math_numpy.py, run_tests.py, brief	Systems Architect and integration

7. How to run

- cd 04_Systems_Architect, then python verify_math_numpy.py (NumPy only).
- pip install torch NumPy, then python run_tests.py (26 tests expected).
- python ps04_engine.py --demo all, or open ps04_solution.ipynb in VS Code and Run All.
- Fill example_scores.json with measured scores and run python alignment_tax.py.

8. Squad sign-off

Station role	Name
Systems Architect	Pranay Kumar Reddy Galva
Protocol Officer	ASHWAN LINGALA
Data Specialist	Lakshman Kumar Tata
Security Sentinel	Bharath Reddy Vupperpally

9. Verification status and limits

- Verified: the maths. verify_math_numpy.py ran and passed: ln 2 at zero margin, gradient matches finite differences, log Z cancels, DPO margin grows with length while SimPO margin stays constant in a simulation.
- Not verified: all PyTorch code (the PyTorch modules, 26 tests, engine, notebook) was written and syntax-checked, but PyTorch could not be installed in the build environment, so none of it has been executed. Run python run_tests.py first and fix anything it reports before presenting.
- The notebook is saved without outputs; results appear only after you run it.
- The benchmark uses a toy language model, synthetic preference pairs and a single seed. It shows the mechanics work, not how a real LLM behaves, and does not prove SimPO beats DPO.
- No real MMLU/GSM8K/HumanEval scores are included; the audit file is a template.
- No GitHub repository was created. Create it from your own account and paste the URL into the notebook sign-off and Section 8.
- The 7.1% to 28.4% consciousness figures come from the mission story; nothing in the code computes them.

10. References

- Rafailov et al. (2023). Direct Preference Optimization.
- Meng, Xia, Chen (2024). SimPO: Simple Preference Optimization with a Reference-Free Reward. NeurIPS 2024.
- Ethayarajh et al. (2024). KTO: Model Alignment as Prospect Theoretic Optimization. ICML 2024.
- Shao et al. (2024). DeepSeekMath (GRPO).
- Zou et al. (2023). Representation Engineering.